

Project 07 (RAG+MCP) — Manual Promotion: Full Reference

Part 1: Manual Dev→Prod Promotion — Steps

1. Fix the `allowed_hosts` design flaw first

`mcp_wrapper.py` originally hardcoded `allowed_hosts` per environment, forcing the shared "frozen" image to be rebuilt every time a new environment's hostname needed adding — defeating true promotion. Fixed by reading a single `ALLOWED_HOST` env var instead, injected per-environment via Terraform.

```
EO
SE

python
allowed_host = os.getenv("ALLOWED_HOST")
security = TransportSecuritySettings(
    allowed_hosts=[allowed_host, f"{allowed_host}.*"],
    enable_dns_rebinding_protection=True,
```

```
EO
SE

hcl
env {
  name = "ALLOWED_HOST"
  value = "rag-pipe-mcp-dev-mcp-server-ncdlcaaczq-ez.a.run.app"
```

2. Fix `create_extension.py`'s hardcoded GRANT role

Same class of bug — the `GRANT ALL ON SCHEMA public TO "..."` statement had dev's SA email literally hardcoded, breaking prod (which has its own different SA). Fixed by deriving the role name from `DB_USER` env var, with `.gserviceaccount.com` stripped:

```
EO
SE

python
db_user = os.getenv("DB_USER").replace(".gserviceaccount.com", "")
# use db_user in the GRANT statement
```

3. Rebuild + push the corrected image to dev's AR repo (once)

```
EO
SE

bash
docker build -t europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest .
docker push europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest
```

This is now the final, environment-agnostic image — never needs rebuilding again for new environments.

4. Update dev's Cloud Run Service + Job to the new image, redeploy dev

```
EO
SE

bash
gcloud run deploy rag-pipe-mcp-dev-mcp-server --image=europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest --region=europe-west4
gcloud run jobs update rag-pipe-mcp-dev-embed-job --image=europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest --region=europe-west4
```

5. Add missing IAM bindings to `iam.tf` for `pipeline_sa`

Discovered that AR writer/reader, `run.admin`, and `iam.serviceAccountUser` (self-grant) were never in Terraform — only added manually via `gcloud` in Iter2. Added permanently to `iam.tf`:

```
EO
SE

hcl
resource "google_project_iam_member" "sa_artifactregistry_writer" {
  project = var.project_id
  role    = "roles/artifactregistry.writer"
  member  = "serviceAccount:${google_service_account.pipeline_sa.email}"
}
resource "google_project_iam_member" "sa_artifactregistry_reader" {
  project = var.project_id
  role    = "roles/artifactregistry.reader"
  member  = "serviceAccount:${google_service_account.pipeline_sa.email}"
}
resource "google_project_iam_member" "sa_run_admin" {
  project = var.project_id
  role    = "roles/run.admin"
  member  = "serviceAccount:${google_service_account.pipeline_sa.email}"
}
resource "google_service_account_iam_member" "sa_self_user" {
  service_account_id = google_service_account.pipeline_sa.name
  role                = "roles/iam.serviceAccountUser"
  member              = "serviceAccount:${google_service_account.pipeline_sa.email}"
}
```

```
EO
SE

bash
terraform apply
```

6. Create prod GCP project + link billing (Console, manual)

7. Checkout prod git branch

```
EO
SE

bash
git checkout -b prod
```

8. Create `prod.tfvars`

EOSE

hcl

project_id = "rag-pipe-mcp-p"

region = "europe-west4"

zone = "europe-west4-a"

ingestion_bucket_name = "rag-pipe-mcp-p-ingestion"

db_tier = "db-f1-micro"

db_name = "ragdb-p"

project_number = "<prod project number>"

9. Point prod's ccloudrun.tf / ccloudrunjob.tf at dev's AR image

EOSE

hcl

image = "europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest"

Delete artefactregistry.tf content on prod's branch entirely — prod provisions no AR repo of its own.

10. New Terraform backend for prod

EOSE

bash

gsutil mb -p rag-pipe-mcp-p -l europe-west4 gs://rag-pipe-mcp-p-terraform-state

terraform init -reconfigure

11. Enable ccloudresourcemanager API manually in prod (chicken-and-egg)

EOSE

bash

gcloud services enable cloudresourcemanager.googleapis.com --project=rag-pipe-mcp-p

12. Apply Terraform for prod

EOSE

bash

terraform apply -var-file="prod.tfvars"

13. Grant cross-project AR access — TWO identities

EOSE

bash

prod's pipeline SA

gcloud artifacts repositories add-iam-policy-binding mcp-server-repo \

--project=rag-pipe-mcp-dev --location=europe-west4 \

--member="serviceAccount:rag-pipe-mcp-p-plpeline@rag-pipe-mcp-p.iam.gserviceaccount.com" \

--role="roles/artifactregistry.reader"

prod's Cloud Run Service Agent (actual identity pulling the image at deploy time)

gcloud artifacts repositories add-iam-policy-binding mcp-server-repo \

--project=rag-pipe-mcp-dev --location=europe-west4 \

--member="serviceAccount:service-<prod-project-number>@serverless-robot-prod.iam.gserviceaccount.com" \

--role="roles/artifactregistry.reader"

14. Re-apply Terraform — prod's Cloud Run Service + Job now succeed

15. Set ALLOWED_HOST for prod (two-pass, first-time-only per environment)

Get prod's real URL from the terraform apply output, add to prod's ccloudrun.tf:

EOSE

hcl

env {

name = "ALLOWED_HOST"

value = "rag-pipe-mcp-p-mcp-server-peiqbzghaq-ez.a.run.app"

EOSE

bash

terraform apply -var-file="prod.tfvars"

No rebuild needed — Terraform updates the env var on the existing service in place.

16. Set prod's own Cloud SQL superuser password

EOSE

bash

gcloud sql users set-password postgres --instance=rag-pipe-mcp-p-pg --password= - password >

17. Run create_extension.py via prod's Cloud Run Job

EOSE

bash

gcloud run jobs execute rag-pipe-mcp-p-embed-job --region=europe-west4 --project=rag-pipe-mcp-p --args=create_extension.py --update-env-vars=POSTGRES_PASSWORD= - password >

18. Upload PDFs to prod's own ingestion bucket

EOSE

bash

gsutil -m cp -r ./data/* gs://rag-pipe-mcp-p-ingestion/

19. Run embed.py via prod's Cloud Run Job

```
bash
gcloud run jobs execute rag-pipe-mcp-p-embed-job --region=europe-west4 --project=rag-pipe-mcp-p
```

20. Test prod's deployed endpoint

```
bash
python3 test_endpoint.py # SERVER_URL pointed at prod's real hostname + /mcp
```

Part 2: Errors Encountered + Fixing Commands

Error	Fix
Backend configuration changed on terraform init/apply	terraform init -reconfigure
Connector ID must follow the pattern ^[a-z][-a-z0-9]{0,23}[a-z0-9]\$	Fixed connector <code>name</code> in <code>cloudrun.tf</code> to a valid lowercase string
Transient DNS blip: dial tcp: lookup <api>.googleapis.com: no such host	Retry <code>terraform apply</code>
Image '...:latest' not found (Cloud Run Service/Job creation)	Build + push the image first, then re- <code>terraform apply</code>
google_sql_database_instance still PENDING_CREATE, apply errored mid-create	Poll <code>gcloud sql instances describe ... --format="value(state)"</code> until <code>RUNNABLE</code> , then <code>terraform import</code> if state lost track, then <code>terraform plan/apply</code>
instanceAlreadyExists (409) on terraform apply	<code>terraform import google_sql_database_instance.rag_pg <PROJECT_ID>/<INSTANCE_NAME></code>
Docker Desktop: Virtualization support not detected	Switched build/push to Cloud Shell instead of local machine
docker: command not found	Confirm Docker Desktop actually running + terminal reopened after install
denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied (Cloud Build trigger push step)	Added <code>artifactregistry.writer</code> to <code>pipeline_sa</code> in <code>iam.tf</code> , <code>terraform apply</code>
250 instance(s) is allowed per prediction. Actual: 1243 (embed.py)	Batch <code>vectorstore.add_documents()</code> calls, <code>batch_size=250</code> initially
input token count is 59953 but the model supports up to 20000	Reduced <code>batch_size</code> to 25 (count-based batching alone insufficient — token limit also applies)
permission denied to create extension "vector" ... Must be superuser	Reset <code>postgres</code> password, ran <code>create_extension.py</code> via Cloud Run Job as superuser
permission denied for schema public	Extended <code>create_extension.py</code> to also <code>GRANT ALL ON SCHEMA public TO "<stripped_sa_email>"</code>
role "..." does not exist on the GRANT step	<code>create_extension.py</code> had the wrong/stale/hardcoded SA email — fixed to use <code>os.environ["DB_USER"]</code> (stripped), rebuilt/pushed/updated job, re-ran
password authentication failed for user "postgres"	Superuser password was never actually reset before running <code>create_extension.py</code> — ran <code>gcloud sql users set-password</code> first
Cloud Run Job kept running an old image despite rebuild	<code>gcloud run jobs update --image=...</code> — jobs don't auto-pick-up <code>:latest</code> on <code>execute</code> , unlike Services
Google Cloud Run Service Agent ... must have permission to read the image (cross-project)	Grant <code>artifactregistry.reader</code> to <code>service-<project-number>@serverless-robot-prod.iam.gserviceaccount.com</code> on the source project's AR repo — separate identity from the app's own SA
MCPError: Server returned an error response / timeout on initialize	<code>allowed_hosts</code> rejecting the request — hostname missing or malformed (had <code>https://</code> prefix, which never matches the raw <code>Host</code> header)
Digest mismatch investigation between running revision and :latest	<code>docker buildx imagetools inspect ...:latest</code> — confirmed manifest-list vs. platform-manifest digest difference, not an actual stale deploy
Unable to remove Service Networking Connection ... Producer services still using this connection (terraform destroy)	Genuine GCP propagation delay after Cloud SQL deletion; resolved via full project deletion: <code>gcloud projects delete rag-pipe-mcp-prod</code>
Backend initialization required after branch switch	<code>terraform init -reconfigure</code>
Cloud Build trigger not found via gcloud builds triggers describe	Trigger was region-scoped (<code>europe-west4</code>), not global — checked via Console, or <code>--region=europe-west4</code> flag
Cloud Build push step PERMISSION_DENIED under trigger's own SA (pipeline_sa)	Added the same AR writer/reader roles to <code>iam.tf</code> (trigger runs as <code>pipeline_sa</code> a user-managed SA per org policy)
Terminal command duplication/mangling on paste (--include-tagsgcloud...)	Retype/rerun the command cleanly

Part 3: Why Terraform + Cloud Deploy Manifest Together Is Ill-Advised

Terraform's `google_cloud_run_v2_service` resource and Cloud Deploy's manifest-based deployment (Scaffold + Cloud Run service YAML) are **two independent systems that each expect to be the sole owner of the deployed Cloud Run service's state**.

Terraform tracks the service's config (image, env vars, SA, scaling) in its own state file and reconciles any drift back to what's declared in `.tf` files on every `apply`.

Cloud Deploy's manifest is a separate declaration of largely the same thing (image, env vars, ports), applied directly to Cloud Run by Cloud Deploy's own release mechanism, with no awareness of Terraform's state at all.

If both are used: whichever one deploys most recently silently overwrites the other's configuration on the live service, while neither system's own state file reflects what is actually running. This produces exactly the kind of "why doesn't my fix work" debugging trap this project already hit multiple times with far simpler causes (stale `:latest` digests, commit/build mismatches) — except here the disagreement is structural, not transient, and won't resolve by retrying or waiting.

Additionally, provisioning Cloud Deploy pipelines themselves is not commonly done via Terraform at all (no clean, first-class resource in most setups) — it's normally managed via `clouddeploy.yaml` + `gcloud deploy apply`, which is itself a second, separate IaC-adjacent mechanism running alongside Terraform, rather than through it.

Given the project's actual goal — a working, correctly-gated dev→prod promotion — manual `gcloud run deploy` (already fully Terraform-owned, no competing system) achieves the same practical outcome (a human-gated promotion step) without this ownership conflict.

Part 4: If Cloud Run Were Provisioned via Cloud Deploy Manifest Instead of Terraform

This describes the alternative architecture — not what was implemented — for reference if a future iteration wants to commit fully to Cloud Deploy instead of Terraform for the Cloud Run layer specifically.

1. Remove Cloud Run resources from Terraform

Delete `google_cloud_run_v2_service.mcp_server` and `google_cloud_run_v2_job.embed_job` from `.tf` files (and their state, via `terraform state rm`). Terraform continues to own everything else (VPC, Cloud SQL, IAM, Artifact Registry, buckets).

2. Create a `scaffold.yaml`

Describes how to render/deploy the service, referencing the manifest file(s):

```
yaml
apiVersion: scaffold/v4beta6
kind: Config
manifests:
  rawYaml:
  - service.yaml
deploy:
  cloudrun: {}
```

3. Create the Cloud Run service manifest (`service.yaml`)

Native-style manifest defining image, env vars, SA, port — effectively re-declaring what `cloudrun.tf` used to define:

```
yaml
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: mcp-server
spec:
  template:
    spec:
      serviceAccountName: PLACEHOLDER_SA
      containers:
      - image: mcp-server
        ports:
        - containerPort: 8080
        env:
        - name: PROJECT_ID
          value: PLACEHOLDER_PROJECT_ID
        - name: ALLOWED_HOST
          value: PLACEHOLDER_ALLOWED_HOST
      # ... remaining env vars matching what cloudrun.tf declared
```

4. Register the delivery pipeline + targets (as already done)

```
bash
gcloud deploy apply --file=clouddeploy.yaml --region=europe-west4 --project=rag-pipe-mcp-dev
```

5. Grant Cloud Deploy's service agent Cloud Run deploy permissions on both projects

```
bash
gcloud projects add-iam-policy-binding rag-pipe-mcp-dev --member="serviceAccount:service-<dev-project-number>@gcp-sa-clouddeploy.iam.gserviceaccount.com" --role="roles/clouddeploy.jobRunner"
gcloud projects add-iam-policy-binding rag-pipe-mcp-p --member="serviceAccount:service-<dev-project-number>@gcp-sa-clouddeploy.iam.gserviceaccount.com" --role="roles/clouddeploy.jobRunner"
```

6. Grant `pipeline_sa` release-creation permission

```
hcl
resource "google_project_iam_member" "sa_clouddeploy_releaser" {
  project = var.project_id
  role    = "roles/clouddeploy.releaser"
  member  = "serviceAccount:${google_service_account.pipeline_sa.email}"
}
```

7. Update `cloudbuild.yaml` to create a release with an explicit `--source`

```
yaml
name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
entrypoint: gcloud
args:
  - 'deploy'
  - 'releases'
  - 'create'
  - 'release-$SHORT_SHA'
  - '--delivery-pipeline=rag-pipe-mcp-pipeline'
  - '--region=europe-west4'
  - '--source=./projects/RAG_PIPELINE_PROM/deploy'
  - '--images=mcp-server=europe-west4-docker.pkg.dev/$PROJECT_ID/mcp-server-repo/mcp-server:latest'
```

8. Push to dev branch — Cloud Build creates the release, Cloud Deploy auto-deploys to dev target

9. Promote to prod once dev is validated

```
bash
gcloud deploy releases promote --release=release-`sha` --delivery-pipeline=rag-pipe-mcp-pipeline --region=europe-west4 --to-target=prod
```

Ongoing consideration

`ALLOWED_HOSTS` (and any other per-environment value) now has to be templated into the manifest per-target rather than injected via Terraform's per-branch tfvars — Cloud Deploy supports this via [render-time variable substitution or per-target manifest overrides], which would need to be set up as an additional piece of this configuration, not something carried over automatically from the Terraform-based approach.